Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «АПО ЭВМиС»
Тема: «**Изучение аппаратно-программной архитектуры процессоров семейства Intel – обработка символьных данных**»

Выполнил:
Студент 4-го курса
Группы АС-66
Гончерёнок К.А.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, практическая работа с подсистемой регистровой и оперативной памяти

Задание: Дан текст – непустая последовательность не длиннее ста символов. Признаком конца ввода является точка, в сам текст точка не входит. Проверить, удовлетворяет ли текст заданному условию. Если условие выполнено, преобразовать текст по указанному в варианте правилу; в противном случае выдать соответствующее сообщение. Преобразованный текст напечатать. Проверяемое условие и правила обработки текста определяются конкретным вариантом задания. Если введенная последовательность символов не является текстом, преобразовывать ее не нужно, а следует напечатать соответствующее сообщение. Ввод текста, проверка условия, обработка текста и печать результата должны выполняться последовательно, отдельными частями программы. 2. Используя знания и навыки, полученные в результате выполнения лабораторной работы №1, а также изучив дополнительные теоретические материалы, приведенные в методическом указании, написать программу на ассемблере, реализующую обработку текста согласно варианту, выданному преподавателем.

Вариант задания: 4) Текст содержит не менее трех латинских букв.

Правило преобразования: 4) Заменить каждую прописную латинскую букву следующей за ней по алфавиту, букву Z менять на A.

Текст программы:

```
.
bits 64

section .data
    prompt_in   db 'Vvedite tekst (do 100 simvolov, konec - tochka):', 10
    prompt_len  equ $ - prompt_in
    cond_text   db 'Uslovie: ne menee treh latinskih bukv.', 10
    cond_len    equ $ - cond_text
    msg_ok      db 'Uslovie vypolneno. Preobrazovanie:', 10
    ok_len      equ $ - msg_ok
    msg_fail    db 'Uslovie NE vypolneno.', 10
    fail_len    equ $ - msg_fail
    msg_empty   db 'Tekst ne vveden.', 10
    empty_len   equ $ - msg_empty
    msg_result  db 'Rezultat:', 10
    result_len  equ $ - msg_result
    newline     db 10
section .bss
    text        resb 101
section .text
global _start
_start:
    ; Prompt
    mov eax, 1
    mov edi, 1
    mov rsi, prompt_in
    mov edx, prompt_len
    syscall
    ; Read at most 101 bytes so we can safely retain the first 100.
    xor eax, eax
    xor edi, edi
    mov rsi, text
    mov edx, 101
    syscall
    test rax, rax
```

```
    jle .empty

    ; Find the end marker and retain at most 100 characters.
    mov r8, rax
    xor ecx, ecx
.scan:
    cmp rcx, r8
    jae .text_ready
    cmp rcx, 100
    jae .text_ready
    mov al, [text + rcx]
    cmp al, '.'
    je .text_ready
    cmp al, 10
    je .text_ready
    cmp al, 13
    je .text_ready
    inc rcx
    jmp .scan

.text_ready:
    mov r10, rcx
    test rcx, rcx
    jz .empty


    mov eax, 1
    mov edi, 1
    mov rsi, cond_text
    mov edx, cond_len
    syscall

    ; Count Latin letters in the retained text.
    xor r9d, r9d
    xor ecx, ecx
.count_loop:
    cmp rcx, r10
    jae .count_done
    mov al, [text + rcx]
    cmp al, 'A'
    jb .count_lower
    cmp al, 'Z'
    jbe .is_letter
.count_lower:
    cmp al, 'a'
    jb .count_next
    cmp al, 'z'
    ja .count_next
.is_letter:
    inc r9d
.count_next:
    inc rcx
    jmp .count_loop
.count_done:
    cmp r9d, 3
    jb .condition_failed

    ; Advance Latin letters, preserving case and wrapping at the alphabet end.
    xor ecx, ecx
.transform_loop:
    cmp rcx, r10
    jae .transformed
    mov al, [text + rcx]
    cmp al, 'A'
    jb .check_lower
```

```
    cmp al, 'Z'
    ja .check_lower
    cmp al, 'Z'
    jne .transform_increment
    mov byte [text + rcx], 'A'
    jmp .transform_next
.check_lower:
    cmp al, 'a'
    jb .transform_next
    cmp al, 'z'
    ja .transform_next
    cmp al, 'z'
    jne .transform_increment
    mov byte [text + rcx], 'a'
    jmp .transform_next
.transform_increment:
    inc al
    mov [text + rcx], al
.transform_next:
    inc rcx
    jmp .transform_loop
.transformed:
    mov eax, 1
    mov edi, 1
    mov rsi, msg_ok
    mov edx, ok_len
    syscall
    mov eax, 1
    mov edi, 1
    mov rsi, msg_result
    mov edx, result_len
    syscall
    mov eax, 1
    mov edi, 1
    mov rsi, text
    mov rdx, r10
    syscall
    mov eax, 1
    mov edi, 1
    mov rsi, newline
    mov edx, 1
    syscall
    jmp .exit

.condition_failed:
    mov eax, 1
    mov edi, 1
    mov rsi, msg_fail
    mov edx, fail_len
    syscall
    jmp .exit

.empty:
    mov eax, 1
    mov edi, 1
    mov rsi, msg_empty
    mov edx, empty_len
    syscall

.exit:
    mov eax, 60
    xor edi, edi
    syscall
```

Вывод: Я изучил программную модель микропроцессора, получил

практический опыт в работе с подсистемой регистровой и оперативной памяти.